DevSecOps

Mikel Egaña Aranguren

mikel-egana-aranguren.github.io

mikel.egana@ehu.eus

Universidad del País Vasco
Euskal Herriko Unibertsitatea
BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

DevSecOps

https://doi.org/10.5281/zenodo.4302267

https://github.com/mikel-egana-aranguren/EHU-SGSSI-01



DevSecOps

- Integra la seguridad (Sec) en el ciclo de vida del software (Dev) y en las operaciones (Ops)
- Desarrolladores, operaciones y seguridad comparten la responsabilidad
- Automatiza comprobaciones para obtener feedback explícito, temprano y continuo

Objetivos y principios

- Incorporar seguridad desde el diseño, no solo antes de publicar
- Automatizar controles repetibles dentro del flujo de trabajo
- Aplicar mínimo privilegio y proteger datos y credenciales
- Tratar los hallazgos como feedback para mejorar continuamente

Seguridad en el ciclo de vida

- Planificación: requisitos de seguridad y análisis de amenazas
- Desarrollo: revisión de código y análisis automatizados
- Integración y entrega: validación de dependencias y artefactos
- Despliegue y operación: configuración segura, monitorización y respuesta

Shift left y feedback continuo

Detectar problemas cerca del momento en que se introduce el cambio reduce el coste de corregirlos.

- Ejecutar comprobaciones en el entorno de desarrollo y en cada cambio
- Mostrar resultados claros y priorizados dentro del flujo de trabajo
- Mantener controles también en producción: la seguridad no termina al desplegar

Requisitos y modelado de amenazas

- Identificar activos, usuarios, servicios y límites de confianza
- Describir amenazas y riesgos relevantes para el sistema
- Definir medidas de mitigación y convertirlas en requisitos comprobables
- Revisar el análisis cuando cambian la arquitectura o las dependencias

Seguridad del código

- Revisión de código y criterios de aprobación compartidos
- SAST (Static Application Security Testing): analizar el código fuente sin ejecutar la aplicación
- DAST (Dynamic Application Security Testing): evaluar la aplicación mientras está en ejecución
- Escaneo de secretos para evitar publicar credenciales

Dependencias y cadena de suministro

- Mantener un inventario de componentes y versiones (SBOM)
- Analizar dependencias y priorizar vulnerabilidades según el riesgo
- Actualizar componentes y verificar el origen de los artefactos
- Usar repositorios y procesos de construcción controlados

SBOM: Software Bill of Materials

- Documentar todos los componentes de software utilizados en un proyecto
- Facilitar la trazabilidad y el control de vulnerabilidades
- Mejorar la transparencia y la gestión del riesgo
- Facilitar la auditoría y el cumplimiento normativo

SBOM: Software Bill of Materials

- CycloneDX: formato estándar para la representación de SBOM (OWASP)
- Mikel Egaña Aranguren, Jesualdo Tomás Fernández-Breis, Bidane Leon Balentzia, Markus Rompe, Alexander García Castro. A comprehensive view of software vulnerability risks through enterprise knowledge graphs. Computers & Security 2026: 163 (104815)

Seguridad en CI/CD

- CI/CD: Continous Integration / Continuous Delivery
- Proteger ramas, revisiones y permisos de ejecución
- Limitar los permisos de cada tarea al mínimo necesario
- Evitar credenciales persistentes y no incluir secretos en el código
- Validar y conservar la procedencia de los artefactos publicados

Contenedores e infraestructura como código

- Definir la infraestructura como código y versionarla (Ansible examples)
- Usar imágenes base mantenidas y reducir componentes innecesarios
- Ejecutar servicios con permisos mínimos y configuración explícita
- Analizar imágenes y revisar cambios de infraestructura antes del despliegue
- Gestionar secretos fuera de imágenes y ficheros versionados

Despliegue y operación

- Aplicar configuraciones seguras y controlar los cambios en producción
- Registrar eventos relevantes y proteger los registros
- Monitorizar servicios y definir procedimientos de respuesta a incidentes
- Corregir vulnerabilidades y aprender de los incidentes

Medición y mejora continua

- Medir tiempos de detección y corrección, además de hallazgos pendientes
- Revisar falsos positivos y ajustar los controles
- Usar los resultados para priorizar mejoras, no solo para bloquear entregas
- Compartir responsabilidades y aprendizajes entre los equipos